

Manifest: Product Requirement Document (PRD) v3.0

Tagline: *Where Intention Becomes Reality.* (의도가 현실이 되는 곳)

Identity: AI-Native Orchestration IDE (Headless IDE & Sidecar)

1. 개요 (Executive Summary)

1.1. 배경

AI 코딩 도구의 발달로 코드 생성 속도는 비약적으로 향상되었으나, 프로젝트 규모가 커질수록 인간 개발자가 전체 구조를 장악하지 못하는 '**코드 문맹(Code Blindness)**'과 기획서-코드 간의 '**괴리(Drift)**'가 발생함.

1.2. 제품 정의

Manifest는 1인 개발자 및 아키텍트가 복잡한 시스템을 직관적으로 지휘하고 통제할 수 있도록 설계된 **지휘 관제소형 IDE**임. 실제 코딩은 외부 IDE(VS Code 등)에서 수행하되, Manifest는 설계(의도), 구조(청사진), 실행(검증)을 한곳에서 통합 관리하는 **Headless IDE** 전략을 취함.

2. 제품 비전 및 핵심 철학

- Conductor Experience:** 개발자를 코드를 치는 '노동자'에서 프로젝트 전체를 지휘하는 '지휘자'로 격상 시킴.
- Visual Truth:** 기획(Architect) - 구조(Blueprint) - 결과(Inspector)를 실시간 동기화하여 시각적으로 검증 가능한 진실을 제공.
- Greenfield First:** 백지상태에서 기술 부채 없는 완벽한 아키텍처로 시작할 수 있는 최적의 환경 지원.
- Sidecar Architecture:** 기존 개발 환경을 대체하는 것이 아니라, 강력한 보조 엔진으로서 병행 구동됨.

3. 통합 관제 인터페이스 (The 5-View Workspace)

3.1. 레이아웃 구조 (Unified Control View)

- 좌측 (Mission Control):** 활성 태스크 리스트 및 **Continuation Enforcer (Agent To-Do)** 상시 노출.
- 중앙 (Unified Workspace):**
 - * Left Side (Design):** Architect(기술 명세) 또는 Blueprint(시스템 지도) 표시.
 - Right Side (Inspector):** 라이브 UI 프리뷰 또는 데이터 실행 흐름(Trace)을 실시간 매칭하여 표시.
- 하단 (Command Center):** 오케스트레이터 및 스쿼드별 상세 로그/대화 탭.

3.2. 핵심 뷰 상세

- Architect View (의도):** 자연어 대화를 Feature/Requirement 단위의 구조화된 명세로 변환 및 관리.
- Blueprint View (구조):** Client-Server-Data 3-Zone 기반의 논리적 연결도 시각화. 미구현 요소는 'Ghost Node'로 표시.

3. **Inspector View (검증):** 구현된 화면과 피그마 디자인 비교(Visual Diff) 및 함수 단위 실행 데이터 (Input/Output) 추적.
4. **Mission Control (관리):** 에이전트의 작업 단계(Plan-Code-Test) 승인 게이트 및 실시간 태스크 현황 관제.
5. **History View (이력):** 태스크와 연동된 Git 커밋 그래프. 특정 작업 시점으로 설계와 코드를 동시에 롤백.

4. 핵심 에이전트 및 관리 로직

4.1. Centralized Orchestration (중앙 집권식 관리)

- **Orchestrator Engine:** 모든 태스크 상태와 데이터의 'Single Source of Truth'.
- **Logic:** 개별 에이전트(Coder, Tester 등)는 오직 오케스트레이터에게 보고만 수행하며, 상태 갱신 및 UI 반영은 오직 오케스트레이터 엔진을 통해서만 일어남.

4.2. Continuation Enforcer (연속성 강제)

- **Agent To-Do:** 에이전트가 수행해야 할 세부 체크리스트를 사이드바에 명시.
- **Next Action:** AI가 컨텍스트를 잃거나 멈췄을 때, 다음에 무엇을 해야 할지를 단 한 줄로 정의하여 즉시 복구 가능하게 함.

4.3. Drift Resolver (괴리 해결사)

- 설계 문서(Architect)와 실제 코드(Raw Code) 간의 불일치를 주기적으로 감지하여 경고를 띄우고, "설계 수정" 또는 "코드 수정" 중 하나를 선택하도록 강제.

5. MVP 개발 전략 (TUI to Electron)

Manifest는 개발자 친화적인 환경에서 시작하여 점진적으로 풍부한 시각화로 진화함.

5.1. Phase 1: TUI (Terminal User Interface) - MVP

- **목표:** 핵심 에이전트 로직 및 5-View 레이아웃의 사용성 고속 검증.
- **기술:** Python `Textual` 라이브러리 사용.
- **UX:** 가볍고 빠르며, 터미널 명령과 에이전트 로그를 실시간으로 확인하는 개발자용 환경 구축.

5.2. Phase 2: Electron Desktop App

- **목표:** 고도화된 시각화(Diagram, UI Preview) 및 완성도 높은 GUI 제공.
- **기술:** Electron + React + TailwindCSS.
- **연결:** Phase 1에서 검증된 **Python Core** 엔진을 **Sidecar**로 내장.
- **통신:** HTTP 대신 **IPC (Standard I/O Pipe)** 방식을 사용하여 포트 충돌 및 보안 이슈 없는 초고속 데이터 교환 실현.

6. 기술적 요구사항

- **Core Engine:** Python 기반의 AI 오케스트레이션 (LangGraph, Aider Core 활용).

- **File Watcher:** 로컬 파일 변경을 실시간 감지하여 Blueprint 및 Inspector 갱신.
- **Shadow Workspace:** 사용자 원본 코드를 오염시키지 않는 샌드박스 검증 환경 구축.
- **AST Parser:** 코드의 논리 구조를 분석하여 Blueprint 다이어그램 자동 생성.

7. 향후 과제

1. **Human-in-the-loop 고도화:** 에이전트 작업 중 사용자 개입 인터페이스 최적화.
2. **멀티 프로젝트 관리:** 여러 마이크로서비스 프로젝트 간의 의존성 시각화 지원.
3. **설계 자동화:** 기획 의도 입력 시 최적화된 DB 스키마 및 아키텍처 자동 제안.